Aloia PDF Extractor

User Guide

Version 1.0

March 2026

Classification: Internal Use Only

Table of Contents

1. Overview
2. Prerequisites & Installation
3. Getting Started
4. Step 1 – Define Extraction Fields
5. Step 2 – Upload PDF Documents
6. Step 3 – Review & Download Results
7. Cross-Reference (VLOOKUP) Workflow
8. API Reference
9. Configuration & Environment
10. Troubleshooting
11. Appendix – Field Schema Examples

Confidential

1. Overview

Aloia PDF Extractor is an AI-powered web application that extracts structured data from PDF documents using Anthropic's Claude AI. It enables business users to define custom data fields, upload one or more PDFs, and receive the extracted information as a downloadable CSV file.

Key capabilities include:

- **AI-Powered Extraction** – Uses Claude Sonnet to intelligently read and interpret PDF content.
- **Flexible Schema Definition** – Define what to extract using a visual form builder, JSON schema, or plain English.
- **Batch Processing** – Upload and process multiple PDFs in a single operation.
- **Cross-Reference / VLOOKUP** – Match extracted data against a lookup dataset using fuzzy matching and export the enriched results.
- **CSV Export** – All results are downloadable as standard CSV files compatible with Excel, Google Sheets, and other tools.

2. Prerequisites & Installation

System Requirements

Requirement	Details
Runtime	Node.js 18 or later
Package Manager	npm (included with Node.js)
API Key	Valid Anthropic API key with access to Claude Sonnet
Browser	Any modern browser (Chrome, Edge, Firefox, Safari)

Installation

Clone the repository and install dependencies:

```
git clone <repository-url>
cd aloia-pdf-extractor
npm install
```


Confidential

Environment Configuration

Create a file named `.env.local` in the project root with the following content:

```
ANTHROPIC_API_KEY=your_api_key_here
```

> **Security Notice:** Never commit your API key to version control. The `.env.local` file is excluded from Git by default. Store keys securely and rotate them according to your organization's security policy.

Starting the Application

Command	Description
`npm run dev`	Start in development mode (hot reload enabled)
`npm run build`	Create a production build
`npm run start`	Run the production build

Once started, open `http://localhost:3000` in your browser.

3. Getting Started

The application guides you through a step-by-step workflow. A progress indicator at the top of the screen shows your current position. The basic extraction flow consists of three steps:

1. **Define Fields** – Specify what data you want to extract.
2. **Upload PDFs** – Provide the documents to process.
3. **Review Results** – Preview and download the extracted data as CSV.

An optional three-step extension enables cross-referencing extracted data against a second dataset (Steps 4–6).

4. Step 1 – Define Extraction Fields

Before uploading documents, you must tell the application what data to extract. Three input modes are available, accessible via tabs at the top of the form:

Confidential

4.1 Field Builder (Recommended for Most Users)

The visual form builder lets you add fields one at a time. For each field, provide:

- **Field Name** – The column header for this data point (e.g., `invoice_number`).
- **Description / Hint** – Guidance for the AI on where or how to find this value (e.g., *"Usually at the top right of the first page"*).

Click **Add Field** to append a new field. Click the **Remove** button next to any field to delete it.

4.2 JSON Schema

For advanced users or automated workflows, paste a JSON array defining the fields:

```
[
  { "name": "vendor_name", "description": "Company name of the vendor" },
  { "name": "invoice_date", "description": "Date the invoice was issued" },
  { "name": "total_amount", "description": "Total amount due" }
]
```

4.3 Natural Language

Describe what you need in plain English. The AI will interpret your instructions and extract accordingly. This mode is ideal for complex or variable documents.

```
Extract the invoice number, vendor name, invoice date,
each line item with its description, quantity, unit price,
and the total amount due.
```

> **Tip:** Be as specific as possible with field descriptions. The more context you provide, the more accurate the extraction results will be.

5. Step 2 – Upload PDF Documents

After defining your schema, proceed to the upload step:

- **Drag and drop** one or more PDF files onto the upload zone, or **click** to browse your file system.
- The selected files are listed below the upload zone. Use the **Remove** button to deselect any file before processing.
- Click **Extract** to begin processing.

A progress bar tracks the extraction. Processing time depends on the number and complexity of the uploaded documents.

Confidential

> **Note:** Each PDF is sent to the AI individually. The results from all documents are merged into a single output CSV. A `_source_file` column identifies which PDF each row originated from.

6. Step 3 – Review & Download Results

Once extraction is complete, a preview table displays the first 20 rows of extracted data. From here you can:

Action	Description
Download CSV	Save the full dataset as a CSV file to your local machine.
Cross-Reference	Proceed to the cross-reference workflow (Steps 4–6) to match this data against a lookup dataset.
Start Over	Reset the application and begin a new extraction.

7. Cross-Reference (VLOOKUP) Workflow

The cross-reference feature lets you match your extracted data against a second dataset — similar to a VLOOKUP in spreadsheet applications. This is useful for enriching extracted records with data from a reference source.

4 Define & Upload Lookup Data

Define the schema for your lookup documents using the same three modes (Field Builder, JSON, or Natural Language), then upload the lookup PDFs. The application extracts data from these documents to create the lookup dataset.

5 Configure the Match

Configure how the two datasets should be joined:

- **Source Column** – Select the column from your primary extraction to match on.
- **Lookup Column** – Select the corresponding column from the lookup dataset.
- **Return Columns** – Choose which columns from the lookup dataset to include in the output. Use *Select All* or *Clear* for convenience.

Click **Run Match** to execute.

Confidential

6 Match Results

The results table shows all matched rows. The match count is displayed above the table. Available actions:

- **Download Matches CSV** – Export the enriched dataset.
- **Adjust Match** – Return to Step 5 to modify matching criteria.
- **Start Over** – Reset the entire workflow.

> **How Matching Works:** The application performs case-insensitive substring matching. A source value matches a lookup value if the source value is contained within the lookup value. A single source row may match multiple lookup rows (one-to-many relationship).

8. API Reference

For integration and automation purposes, the application exposes two API endpoints.

8.1 POST /api/extract

Extracts structured data from uploaded PDFs.

Parameter	Type	Description
`pdfs`	File[]	One or more PDF files (multipart/form-data)
`schema`	string	Field definitions or natural language instructions
`mode`	string	Schema type: `form`, `json`, or `nl`

Response: `text/csv` – CSV content with extracted data.

8.2 POST /api/cross-reference

Performs fuzzy matching between two datasets.

Confidential

Parameter	Type	Description
`sourceRows`	Object[]	Array of source data objects
`lookupRows`	Object[]	Array of lookup data objects
`sourceCol`	string	Column name to match from source
`lookupCol`	string	Column name to match in lookup
`returnCols`	string[]	Columns to include from lookup

Request format: `application/json`

Response: `text/csv` – CSV with matched and enriched records.

9. Configuration & Environment

Environment Variables

Variable	Required	Description
`ANTHROPIC_API_KEY`	Yes	API key for authenticating with the Anthropic Claude API

AI Model

The application uses **Claude Sonnet 4.6** (`claude-sonnet-4-6-20250514`) for document analysis. The maximum output token limit is set to **16,384 tokens** per request.

Confidential

Project Structure

```
aloia-pdf-extractor/
├── app/
│   ├── api/
│   │   ├── extract/route.ts         # Extraction endpoint
│   │   └── cross-reference/route.ts # Cross-reference endpoint
│   ├── page.tsx                     # Application entry point
│   ├── layout.tsx                   # Root layout
│   └── globals.css                  # Global styles
├── components/
│   ├── PDFExtractor.tsx             # Main workflow component
│   ├── SchemaBuilder.tsx            # Field definition UI
│   └── MatchConfigurator.tsx        # Cross-reference configuration
├── lib/
│   ├── pdfExtractor.ts             # Prompt building & parsing
│   └── fuzzyMatch.ts               # Matching logic
├── .env.local                       # Environment variables (not in Git)
└── package.json
```

10. Troubleshooting

Issue	Possible Cause	Resolution
Extraction returns an error	Invalid or missing API key	Verify `ANTHROPIC_API_KEY` is set correctly in `.env.local` and restart the server.
Empty or incomplete results	Vague field descriptions	Provide more specific descriptions or hints for each field. Use natural language mode for complex documents.
Cross-reference returns no matches	No substring overlap between source and lookup values	Verify that the selected columns contain comparable data. The match is case-insensitive and uses substring containment.
Application fails to start	Missing dependencies	Run `npm install` and ensure Node.js 18+ is installed.
Large PDF processing is slow	Document complexity / API latency	Consider splitting very large PDFs into smaller sections before uploading.
CSV has unexpected formatting	Special characters in extracted data	The application handles CSV escaping automatically. If issues persist, check the source PDF content.

Confidential

11. Appendix – Field Schema Examples

Invoice Processing

```
[
  { "name": "invoice_number", "description": "Unique invoice identifier" },
  { "name": "vendor_name", "description": "Name of the issuing vendor/company" },
  { "name": "invoice_date", "description": "Date the invoice was issued (YYYY-MM-
DD)" },
  { "name": "due_date", "description": "Payment due date" },
  { "name": "line_item", "description": "Description of each line item" },
  { "name": "quantity", "description": "Quantity for each line item" },
  { "name": "unit_price", "description": "Unit price for each line item" },
  { "name": "total_amount", "description": "Grand total amount due" }
]
```

Employee Records

```
[
  { "name": "employee_id", "description": "Employee ID number" },
  { "name": "full_name", "description": "Employee full name" },
  { "name": "department", "description": "Department or division" },
  { "name": "job_title", "description": "Current job title or role" },
  { "name": "start_date", "description": "Employment start date" },
  { "name": "salary", "description": "Annual salary or compensation" }
]
```

Contract Review (Natural Language)

```
Extract the following from each contract document:
- Party names (all signatories)
- Effective date and termination date
- Contract value or total consideration
- Key obligations for each party
- Governing law jurisdiction
- Any renewal or termination clauses
```